

mfsk-core

WSJT-X のデコーダを、**まるごと Rust** で書き直しました。
デスクトップにも、ブラウザにも、スマホにも、**マイコン**にも。

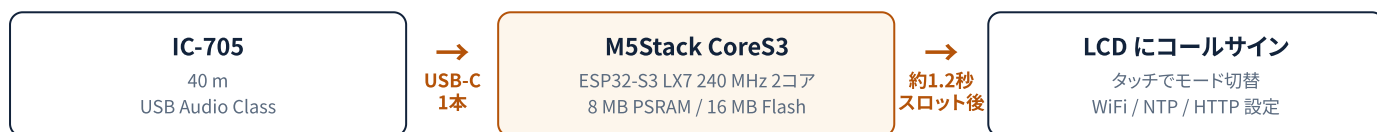
github.com/jl1nie/mfsk-coregithub.com/jl1nie/mfsk-core · crates.io/crates/mfsk-core · GPL-3.0-or-later · JL1NIE

これは何か

FT8・FT4・FST4・WSPR・JT9・JT65・Q65・MSK144 — WSJT 系デジタルモードの復調と変調を、ひとつの純 Rust ライブラリにまとめたものです。DSP、同期相関、LLR、誤り訂正 (LDPC / 畳み込み / Reed-Solomon / QRA)、メッセージ符号を共通の土台として共有し、その上に各モードが載っています。

WSJT-X はこれからも**基準実装**です。各ファイルは移植元の `lib/*.f90` / `lib/*.c` を明記しています。このライブラリの存在理由は WSJT-X を置き換えるのではなく、**Fortran + C + Qt のデスクトップアプリが行けない場所へ、同じアルゴリズムを持っていく**ことです。

ブースで動いているもの



この経路に PC は 1 台も入っていません。復調はすべて CoreS3 の中で完結しています。

6-8 局

1 スロット (15 秒) あたりの FT8 デコード数
2026-08-23、IC-705 実機 40 m

-24 dB

到達した最弱信号
(+8 ~ -24 dB)

4 受信機

1 つのイメージに FT8 / WSPR / FST4 /
無線機不要のデモを同梱、タッチで切替

対応モード

モード	スロット	誤り訂正	メッセージ
FT8	15 s	LDPC(174, 91) + CRC-14	77 bit
FT4	7.5 s	LDPC(174, 91) + CRC-14	77 bit
FST4 -15/-30/-60A/-120/-300	15-300 s	LDPC(240, 101) + CRC-24	77 bit
WSPR	120 s	畳み込み $r=1/2$ K=32 + Fano	50 bit
JT9	60 s	畳み込み $r=1/2$ K=32 + Fano	72 bit
JT65	60 s	Reed-Solomon(63, 12) GF(2 ⁸)	72 bit
Q65 -30A / -60A · · E	30 / 60 s	QRA(15, 65) GF(2 ⁸) + CRC-12	77 bit
MSK144	15 s	LDPC(128, 90) + CRC-13	77 bit

感度は WSJT-X と同等、あるいはそれ以上

モード	実録音での再現率	AWGN 50% 交差
FT8	8/8 · JTDX 18/18	約 -21.6 dB
FT4	6/6	約 -16.9 dB
WSPR	9/9	約 -31.5 dB
JT9	7/7	約 -26.6 dB
MSK144	3/3 (SNR も一致)	約 -5.2 ~ -5.8 dB

「再現率だけ」はベンチマークではない

かつてこの表には再現率しか載っていませんでした。「WSPR 8/8、パリティ達成」と書かれていたデコーダは、**同時に8個の存在しないコールサインを出していました**。いまは誤デコードが常設の列で、上表はいずれも **0**。数字はテストが assert しており、下回れば CI が落ちます。FST4・Q65 を含む全モードは docs/notes/BENCHMARKS.md に。

マルチプラットフォーム — ソースは1つ

行き先	ターゲット	状態と備考
デスクトップ / サーバ	x86_64, aarch64	既定構成。rustfft + rayon。cargo add mfsk-core ですぐ使えます
ブラウザ	wasm32-unknown-unknown	rustfft の WASM SIMD。+simd128 の明示を忘れずに (既定では付きません)
Android / iOS	NDK arm64-v8a ほか	NEON。C ABI 経由で、Fortran ランタイム不要
マイコン	no_std + alloc — ESP32-S3 / RP2350 / Cortex-M	実運用中。fft-extern + fixed-point、FFT は呼び出し側が注入
C / C++ から	staticlib	mfsk-ffi (全モード) / mfsk-ffi-ft8 (FT8 のみ)。ヘッダは cbindgen 生成

差し替えられるのは行き先だけではありません。DSP と FEC 全体がスカラ型トレイトで抽象化され、ホストは f32、マイコンは u16 スペクトログラム + i16 DFT + Q11i16 LLR + 整数 BP — コードは同じ一本です。FFT も外部注入。モードはすべて feature フラグです。

組み込み対応の中身

約150 KB FT8 復調に必要な使用可能 RAM (FFT を呼び出し側が用意する場合)	150–200 KB ライブラリ分のフラッシュ使用量 (実測、アプリのグルー込み)	0 バイト 内部 DRAM の常駐スクラッチ / 要求する extern シンボル
--	---	---

呼び出し側が用意するもの

- FFT の実装ひとつ — fft-extern でファクトリ関数を1つ提供するだけ
- アロケータ — no_std + alloc。スロット単位の作業領域は PSRAM で構いません

求めるのはこの2つだけで、スクラッチバッファの配置に悩む必要はありません。スロットあたりの作業領域: スペクトログラム 約360 KB (PSRAM 可)、候補セット 約120 KB、LDPC BP スクラッチ 約12 KB。

速度 — 同じ整数パイプラインで

Ryzen 9 9900X (固定小数点)	約 6 ms
ESP32-S3 LX7 実機	1.19 s
ESP32 Core2 LX6 実機	約 2.8 s

16局が混み合う WSJT-X 参照録音での実測。差はほぼ純粋な CPU 性能比で、アルゴリズム上のオーバーヘッドはありません。WSPR も 1214秒 → 101.6秒まで詰め、9/9 を維持したまま 120秒スロットに収まりました。

設計 — Protocol トレイト

1つのモードは 実行時に1バイトも占めない型として書かれ、その上に ModulationParams (トーン数・シンボルレート・Gray 写像・GFSK 整形)、FrameLayout (シンボル数・スロット長・同期配置)、Protocol (どの FEC と、どのメッセージ符号を使うか) を実装します。共通パイプラインはすべて P: Protocol にジェネリックで、コンパイル時に各モードへ単相化されます — **vtable なし、動的ディスパッチなし**。抽象化のコストはコンパイル時間だけで、実行時にはゼロです。実測で src/ の約3分の1 (58.9k 行中 18.7k 行) が全モード共通のジェネリックコードです。

これが効くということ: FST4-60A の追加は、ZST 1つへの trait 実装 + レジストリ1行でした (共通コードには一切触れていません)。Fortran では、モード間の「同じであり得たはずの部分」はソースファイル間のコピー＆ペーストとして現れます。ここではそれを型で表現しています。

一緒に作りませんか

- ブラウザで動く FT8 — WASM PWA のフロントエンド (ライブラリ側は wasm32 で動きます)
- Android / iOS アプリ — C ABI は用意済み、JNI の骨組みもあります
- ESP32 以外のマイコン — RP2350、Cortex-M。FFT を1つ渡せば動くはずですが、まだ誰も試していません
- CoreS3 の送信側 — 残るは BLE CI-V、ADIF ログ、送信キーイング
- JT65 の残差 — 自前コーパスでは差ゼロ、実バイナリにはまだ届かず。原因未特定 — 面白い問題です



リポジトリ

Issue も PR も歓迎します。

github.com/jl1nie/mfsk-core

crates.io / docs.rs — mfsk-core (最新リリース v0.9.1、0.10.0 準備中) / GPL-3.0-or-later / ブースで直接どうぞ — JL1NIE 友部 稔
まず読むもの: README.md / LIBRARY.md (ホスト API) / EMBEDDED.md (組み込み・C ABI) / BENCHMARKS.md (本紙の数字の出どころ)。
docs/reference/ は全文書に日本語版があります。